MANUAL TÉCNICO DE COLABORACIÓN · GIT & GITHUB

Guía Metodológica de Trabajo Colaborativo

Proyecto: Desarrollo del Elemento Isoparamétrico 3D de 27 Nodos (Familia Lagrange)

Este documento establece el protocolo estándar paso a paso para colaborar en el desarrollo computacional y matemático del repositorio. Para garantizar la integridad del código, toda modificación, formulación o corrección debe realizarse en **ramas alternas** desde la computadora local y fusionarse a la rama principal (`main`) mediante un **Pull Request (PR)** revisado por el equipo.

1. Ciclo de Vida y Estados de los Archivos en Git

Antes de ejecutar comandos en la consola, es fundamental comprender los cuatro espacios por los que transita cualquier archivo dentro de la arquitectura de Git:

1. **Working Directory (Directorio de trabajo):** Es la carpeta física en el disco duro de tu PC donde abres, editas y guardas tus archivos (scripts `.py`, hojas de cálculo, documentación, etc.).
2. **Staging Area (Área de preparación o índice):** Es una zona intermedia donde seleccionas y empaquetas exactamente qué archivos modificados formarán parte de la próxima versión oficial.
3. **Local Repository (Repositorio local / `.git`):** Es la base de datos oculta en tu máquina que almacena el historial completo de versiones (*commits*) de forma permanente y segura sin depender de internet.
4. **Remote Repository (GitHub):** Es el servidor central en la nube donde se sincronizan los avances de todos los colaboradores del proyecto.

2. Preparación Inicial (Solo la primera vez)

Estos pasos se ejecutan únicamente la primera vez que configuras el proyecto en tu computadora.

Paso 2.1: Aceptar la invitación de colaboración

Ingresa a tu correo electrónico o a tus notificaciones en GitHub y acepta la invitación para unirte al repositorio. Internamente, GitHub vincula tu cuenta de usuario a la lista de acceso del proyecto, otorgándote permisos de lectura y escritura para subir ramas remotas.

Paso 2.2: Clonar el repositorio a tu computadora

Abre la consola **Git Bash**, navega hasta la carpeta donde guardas tus proyectos y ejecuta:

```
git clone https://github.com/yocampoh-spec/Desarrollo-del-elemento-isoparametrico-en-3D-de-27-nodos..git
```

- **¿Qué ocurre internamente?** Git contacta al servidor de GitHub, crea una carpeta con el nombre del proyecto en tu PC, descarga todos los archivos junto con el historial completo de versiones y configura automáticamente el enlace remoto bajo el nombre `origin`.
- **Qué verás en pantalla:** Un reporte de descarga indicando `Cloning into 'Desarrollo-del-elemento-isoparametrico-en-3D-de-27-nodos.'...` seguido del conteo de objetos recibidos al `100%, done`.

Paso 2.3: Entrar al directorio del proyecto

```
cd Desarrollo-del-elemento-isoparametrico-en-3D-de-27-nodos.
```

¿Qué ocurre internamente? La terminal ubica su puntero activo dentro de la carpeta del repositorio. Verás que al final de la línea de comandos en Git Bash aparece el indicador `(main)` en color celeste, confirmando que estás ubicado sobre la rama principal.

3. Flujo de Trabajo Diario para Cada Tarea

> **Regla de oro del repositorio:** Nunca edites archivos ni realices commits estando posicionado directamente sobre la rama `main`. Siempre debes crear una rama alterna para aislar tu desarrollo.

Paso 3.1: Sincronizar tu rama principal antes de empezar

Antes de escribir una sola línea de código o crear una rama nueva, descarga los cambios que otros compañeros hayan fusionado recientemente:

```
git checkout main
git pull origin main
```

- **¿Qué hace `git checkout main`?** Asegura que tu puntero local (`HEAD`) esté situado sobre la rama principal.
- **¿Qué hace `git pull origin main`?** Consulta a GitHub si existen commits nuevos en la nube y los descarga y fusiona automáticamente en tu copia local.
- **Qué verás en pantalla:** Si no hubo cambios nuevos verás `Already up to date.`; si alguien subió avances, verás la lista de archivos actualizados con signos `+` (líneas agregadas) y `-` (líneas eliminadas).

Paso 3.2: Crear y saltar a tu rama alterna de trabajo

Crea una bifurcación específica para la función o corrección que vas a realizar utilizando un nombre claro:

```
git checkout -b feature/nombre-de-tu-tarea
```

- **Ejemplos de nombres recomendados:** `feature/funciones-forma`, `feature/matriz-rigidez-k`, `feature/cuadratura-gauss`, o para correcciones `fix/indices-nodos`.
- **¿Qué ocurre internamente?** El parámetro `-b` ejecuta dos acciones simultáneas: crea una rama idéntica a partir del estado actual de `main` y traslada tu entorno de trabajo hacia ella. Cualquier modificación que hagas a partir de este momento quedará completamente aislada de la rama principal.
- **Qué verás en pantalla:** El mensaje `Switched to a new branch 'feature/nombre-de-tu-tarea'`, y el indicador de tu consola cambiará de `(main)` a `(feature/nombre-de-tu-tarea)`.

Paso 3.3: Modificar los archivos e inspeccionar el estado

Abre tus archivos en tu editor de preferencia (VS Code, Spyder, etc.), realiza las modificaciones, guarda con `Ctrl + S` y regresa a Git Bash para verificar el estado:

```
git status
```

- **¿Qué ocurre internamente?** Git escanea tu carpeta de trabajo y la compara línea por línea contra el último commit registrado en tu historial local.
- **Qué verás en pantalla:** Los archivos modificados aparecerán listados en color **rojo** bajo la sección `Changes not staged for commit`. Si creaste un archivo nuevo, aparecerá en rojo bajo `Untracked files`.

Paso 3.4: Pasar los cambios al Área de Preparación (Staging Area)

Empaqueta los archivos modificados para incluirlos en tu próxima versión:

```
git add .
```

(Si solo deseas subir un archivo específico y dejar otros fuera, escribe el nombre del archivo en lugar del punto: `git add funciones_forma.py`).

- **¿Qué ocurre internamente?** Git toma el estado actual de los archivos modificados y los copia dentro del índice de preparación (*Staging Area*). Si ejecutas nuevamente `git status`, comprobarás que los nombres de los archivos pasaron a color **verde** bajo el título `Changes to be committed`.

Paso 3.5: Registrar los cambios en el historial local (Commit)

Guarda una versión formal en tu repositorio local acompañada de un mensaje descriptivo entre comillas:

```
git commit -m "Implementa deduccion de funciones de forma cuadraticas"
```

- **¿Qué ocurre internamente?** Git toma una instantánea inmutable de todo lo que estaba en verde en el Staging Area, le asigna un código identificador único (hash SHA), registra tu autoría y fecha, y lo almacena en tu carpeta `.git`. En este punto el cambio está seguro en tu PC, pero aún no se ha subido a internet.
- **Qué verás en pantalla:** Una línea resumen indicando el nombre de la rama, el código corto del commit y el número de archivos e inserciones registradas.

Paso 3.6: Subir tu rama alterna a GitHub

Transfiere tu rama local con sus commits hacia el repositorio remoto en GitHub:

```
git push -u origin feature/nombre-de-tu-tarea
```

- **¿Qué ocurre internamente?** Git se conecta a través de internet con `origin` (GitHub), crea una copia exacta de tu rama en la nube y transfiere los paquetes de datos. La bandera `-u` deja vinculada tu rama local con la remota para que en futuros envíos sobre la misma rama solo necesites escribir `git push`.
- **Qué verás en pantalla:** El progreso de compresión y envío al 100%, cerrando con la confirmación `* [new branch] feature/nombre-de-tu-tarea -> feature/nombre-de-tu-tarea`.

4. Integrar Cambios a la Rama Principal (Pull Request)

Para evitar conflictos y mantener control de calidad entre los colaboradores, la fusión hacia la rama principal no se ejecuta desde la consola local, sino mediante la plataforma web de GitHub:

1. Abre el repositorio en tu navegador web. Verás un recuadro amarillo en la parte superior avisando que tu rama fue actualizada recientemente; haz clic en el botón verde **Compare & pull request**.
2. Verifica los selectores de rama en la parte superior para confirmar la dirección de la fusión: `base: main` ← `compare: feature/nombre-de-tu-tarea`.
3. Escribe un título claro y detalla en el cuadro de comentario qué ecuaciones, scripts o módulos modificaste y si verificaste su funcionamiento.
4. Haz clic en **Create pull request**.
5. Tu compañero podrá revisar línea por línea el código en la pestaña **Files changed**. Una vez validado el trabajo en conjunto, se presiona el botón verde **Merge pull request** y luego **Confirm merge**.

¿Qué ocurre internamente al hacer Merge? GitHub une el historial de tu rama alterna dentro de la rama principal `main`, dejando el nuevo código oficialmente integrado para todo el equipo.

5. Limpieza y Sincronización Post-Fusión

Una vez que el Pull Request fue fusionado en la web, tu rama alterna ya cumplió su propósito. Debes actualizar tu computadora y borrar la rama vieja para mantener tu espacio limpio:

```
git checkout main
git pull origin main
git branch -d feature/nombre-de-tu-tarea
```

- **¿Qué ocurre internamente?** Regresas a la rama principal local, descargas la nueva versión oficial que ya contiene tu trabajo fusionado, y eliminas el puntero local de la rama temporal con `git branch -d`. Ningún archivo se pierde porque los cambios ya forman parte de `main`.

6. Tabla de Referencia Rápida de Comandos

Comando en Git Bash	Función y efecto en el repositorio
`git status`	Muestra qué archivos han sido modificados (rojo), cuáles están listos para commit (verde) y en qué rama te encuentras.
`git checkout main`	Te cambia de la rama actual hacia la rama principal `main`.
`git pull origin main`	Descarga e integra en tu computadora los últimos cambios subidos a GitHub en la rama principal.
`git checkout -b <rama>`	Crea una nueva rama alterna a partir de tu ubicación actual y te mueve inmediatamente dentro de ella.
`git add .`	Pasa todos los archivos modificados o nuevos del directorio de trabajo hacia el Área de Preparación (*Staging Area*).
`git commit -m "mensaje"`	Guarda una instantánea permanente en el historial de tu PC con los archivos que estaban en el Staging Area.
`git push -u origin <rama>`	Sube tu rama local y sus commits hacia el repositorio de GitHub en la nube.
`git branch -d <rama>`	Elimina de forma segura una rama de tu computadora una vez que sus cambios ya fueron fusionados en `main`.